МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВЯТСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
Институт математики и информационных систем
Факультет автоматики и вычислительной техники
Кафедра электронных вычислительных машин

Отчёт по лабораторной работе № 2.2
по дисциплине
«Управление данными»
«Основы DML-запросов в PostgreSQL»

Выполнил студент гр. ИВТб-2303-06-00 _____ /Перевозчиков Д.А./
Проверил _____ /Клюкин В. Л./

Киров
2026

1 Цели лабораторной работы

- научиться использовать агрегатные функции;
- научиться использовать операторы и встроенные функции, работать с датами;
- научиться использовать подзапросы и предложения `WITH`, `RETURNING`;
- рассмотреть и выполнить различные планы выполнения запросов;
- выполнить `FORMAT JSON` для выборки из нескольких связанных таблиц;
- выполнить три способа соединения для одной и той же выборки и определить наиболее оптимальный.

2 Задание

Выполнить все примеры из методических указаний для своей предметной области, используя данные и знания, полученные в предыдущих лабораторных работах. В качестве предметной области используется база данных гоночного автоклуба, созданная в лабораторной работе № 1 и заполненная данными в лабораторной работе № 2.1. База данных содержит таблицы `car`, `trainer`, `trainer_car`, `competition` и `achievement`.

3 Решение

3.1 Агрегатные функции и предложение HAVING

Агрегатные функции вычисляют одно значение по множеству входных строк. Для получения результата отдельно по каждой группе строк используется предложение `GROUP BY`. Условия, в которых участвует результат агрегатной функции, записываются в предложении `HAVING`, а не в `WHERE`: `WHERE` фильтрует отдельные строки до группировки, а `HAVING` — уже сформированные группы.

Запрос 1. Подсчёт количества автомобилей в автопарке с помощью `COUNT(*)`.

```
SELECT COUNT(*) AS total_cars
FROM car;
```

Таблица 1 – Результат запроса 1

total_cars
12

В автопарке числится 12 автомобилей, включая автомобили, отправленные в утилизацию.

Запрос 2. Вычисление среднего, минимального, максимального и суммарного значений мощности для автомобилей, не находящихся в утилизации.

```
SELECT
    AVG(horsepower) AS average_horsepower,
    MIN(horsepower) AS min_horsepower,
    MAX(horsepower) AS max_horsepower,
    SUM(horsepower) AS total_horsepower
FROM car
WHERE is_utilized = FALSE;
```

Таблица 2 – Результат запроса 2

average_horsepower	min_horsepower	max_horsepower	total_horsepower
258.8181818181818182	76	510	2847

Средняя мощность рабочих автомобилей составляет около 258,8 л.с., минимальная — 76 л.с., максимальная — 510 л.с.

Запрос 3. Группировка автомобилей по цвету. Для каждого цвета определяется количество автомобилей и их средняя мощность, результат сортируется по убыванию количества.

```
SELECT
    color,
    COUNT(*) AS car_count,
    AVG(horsepower) AS average_horsepower
FROM car
WHERE is_utilized = FALSE
GROUP BY color
ORDER BY car_count DESC;
```

Таблица 3 – Результат запроса 3

color	car_count	average_horsepower
Красный	3	164
Оранжевый	2	265
Белый	2	135
Синий	2	297.5
Серый	1	510
Чёрный	1	450

Запрос 4. Группировка автомобилей по тренерам. Функция `COUNT` считает количество автомобилей у тренера, а `STRING_AGG` собирает их названия в одну строку через запятую. Используется `LEFT JOIN`, чтобы в результат попадали и тренеры без специализаций.

```
SELECT
    t.id,
    t.last_name,
    t.first_name,
    COUNT(tc.car_id) AS car_count,
    STRING_AGG(c.title, ', ') AS cars
FROM trainer t
LEFT JOIN trainer_car tc ON tc.trainer_id = t.id
```

```
LEFT JOIN car c ON c.id = tc.car_id
GROUP BY t.id, t.last_name, t.first_name
ORDER BY car_count DESC;
```

Таблица 4 – Результат запроса 4

id	last_name	first_name	car_count	cars
4	Иванов	Иван	2	ВАЗ 2107, Mitsubishi Lancer Evolution
6	Иванова	Мария	2	Audi RS2, Audi Quattro
2	Пехота	Наталья	2	Audi Quattro, BMW M3
9	Петров	Дмитрий	2	ВАЗ 2107, BMW M3
3	Моралес	Даниэль	2	Peugeot 406, OKA
1	Торрето	Доминик	2	Mazda MX-5 Miata, Toyota Supra
5	Смирнов	Алексей	2	Toyota Supra, Porsche 911
8	Соколова	Анна	2	Mazda MX-5 Miata, Porsche 911

Запрос 5. Поиск тренеров, которые работают с тремя и более автомобилями. Условие на количество записано в предложении `HAVING`, так как оно относится к результату агрегатной функции.

```
SELECT
    t.id,
    t.last_name,
    t.first_name,
    COUNT(tc.car_id) AS car_count
FROM trainer t
JOIN trainer_car tc ON tc.trainer_id = t.id
JOIN car c ON c.id = tc.car_id
WHERE c.is_utilized = FALSE
GROUP BY t.id, t.last_name, t.first_name
HAVING COUNT(tc.car_id) > 2
ORDER BY car_count DESC;
```

Таблица 5 – Результат запроса 5

id	last_name	first_name	car_count
1	Торрето	Доминик	3

Условию удовлетворяет один тренер — Торрето Доминик, который работает с тремя автомобилями.

3.2 Операторы, встроенные функции и работа с датами

Запрос 6. Проверка попадания значения в промежуток с помощью оператора `BETWEEN`: выбираются автомобили мощностью от 400 до 700 л.с., не находящиеся в утилизации.

```
SELECT *
FROM car
WHERE horsepower BETWEEN 400 AND 700
  AND is_utilized = FALSE;
```

Таблица 6 – Результат запроса 6

id	title	photo_url	description	horsepower	production_year	color	suspension_name	is_utilized
7	BMW M3	https://cars.pics/bmw_m3.jpg	Рядная шестёрка, задний привод. Седан, а едет как спорткар.	510	2021	Серый	Adaptive M Suspension	false
10	Porsche 911	https://cars.pics/porsche_911.jpg	Оппозитник сзади. Уже 60 лет не меняют форму кузова. Могут, а зачем?.	450	2022	Чёрный	Sport Suspension	false

Запрос 7. Поиск по шаблону с помощью оператора `LIKE`. Символ `%` в шаблоне заменяет любое количество символов, поэтому выбираются все автомобили, название которых начинается с «Audi».

```
SELECT *
FROM car
WHERE title LIKE 'Audi%';
```

Таблица 7 – Результат запроса 7

id	title	photo_url	description	horsepower	production_year	color	suspension_name	is_utilized
11	Audi RS2	https://cars.pics/audi_rs2.jpg	Универсал, который доводили сами инженеры из Porsche. Первый RS от Audi.	315	1994	Синий	Sport Suspension	false
8	Audi Quattro	https://cars.pics/audi_quattro.jpg	Полный привод и турбонадув. Машина, с которой началась эпоха quattro в ралли.	200	1984	Оранжевый	MacPherson Strut	false

Запрос 8. Формирование полного имени тренера функцией `CONCAT`. Функция `COALESCE` возвращает первый аргумент, отличный от `NULL`, поэтому отсутствующее отчество заменяется пустой строкой. В отличие от оператора `||`, функция `CONCAT` воспринимает `NULL` как пустую строку, и имя не превращается в `NULL`.

```
SELECT
    id,
    CONCAT(last_name, ' ', first_name, ' ',
           COALESCE(middle_name, '')) AS full_name
FROM trainer
ORDER BY id;
```

Таблица 8 – Результат запроса 8

id	full_name
1	Торрето Доминик
2	Пехота Наталья Морская
3	Моралес Даниэль
4	Иванов Иван
5	Смирнов Алексей
6	Иванова Мария
8	Соколова Анна
9	Петров Дмитрий

Запрос 9. Конструкция `CASE` используется для классификации автомобилей по мощности на три категории.

```sql
SELECT
    title,
    horsepower,
    CASE
        WHEN horsepower < 300 THEN 'Низкая мощность'
        WHEN horsepower BETWEEN 300 AND 600 THEN 'Средняя мощность'
        ELSE 'Высокая мощность'
    END AS power_category
FROM car
WHERE is_utilized = FALSE
ORDER BY horsepower DESC;
```

Таблица 9 – Результат запроса 9

title	**horsepower**	**power_category**
BMW M3	510	Средняя мощность
Porsche 911	450	Средняя мощность
Toyota Supra	330	Средняя мощность
Audi RS2	315	Средняя мощность
Nissan Skyline GT-R	280	Низкая мощность
Mazda MX-5 Miata	201	Низкая мощность
Audi Quattro	200	Низкая мощность
Peugeot 406	194	Низкая мощность
Mitsubishi Lancer Evolution	180	Низкая мощность
Volkswagen Beetle	111	Низкая мощность
ВАЗ 2107	76	Низкая мощность

Запрос 10. Вычисление примерного возраста тренеров. Функция `EXTRACT` получает год из текущей даты, результат приводится к целому типу оператором `::INT`, после чего из него вычитается год рождения.

```sql
SELECT
    id,
    last_name,
    first_name,
    birth_year,
    EXTRACT(YEAR FROM CURRENT_DATE)::INT - birth_year AS approximate_age
FROM trainer
ORDER BY birth_year;
```

Таблица 10 – Результат запроса 10

id	last_name	first_name	birth_year	approximate_age
1	Торрето	Доминик	1967	59
3	Моралес	Даниэль	1973	53
9	Петров	Дмитрий	1979	47
4	Иванов	Иван	1985	41
2	Пехота	Наталья	1988	38
6	Иванова	Мария	1990	36
5	Смирнов	Алексей	1992	34
8	Соколова	Анна	1995	31

Запрос 11. Работа с временными интервалами. Из текущей даты вычитается интервал в пять лет, и выбираются соревнования, прошедшие в этот промежуток.

```
SELECT
    id,
    name,
    event_date
FROM competition
WHERE event_date BETWEEN CURRENT_DATE - INTERVAL '5 years' AND CURRENT_DATE
ORDER BY event_date DESC;
```

Таблица 11 – Результат запроса 11

id	name	event_date
11	Кубок Поршня	2023-06-15
12	Гонка Чемпионов	2022-09-03

Запрос 12. Вычитание дат. Разность двух значений типа `date` равна количеству дней между ними.

```
SELECT
    name,
    event_date,
    CURRENT_DATE - event_date AS days_since_competition
FROM competition
WHERE event_date <= CURRENT_DATE
ORDER BY event_date DESC;
```

Таблица 12 – Результат запроса 12

name	event_date	days_since_competition
Кубок Поршня	2023-06-15	1230
Гонка Чемпионов	2022-09-03	1515
Ралли Монте-Карло	2021-01-24	2102
Ралли Финляндия	2020-08-07	2272
Ралли Греция	2019-06-02	2704
Ралли Уэльс	2018-10-05	2944
Ралли Мичиган	2017-07-14	3392
Ралли Испания	2016-10-16	3663
Ралли Марокко	2015-04-26	4202
Ралли Австралия	2014-09-12	4428

3.3 Подзапросы и предложение WITH

Результаты одного запроса можно использовать в другом. Подзапрос может возвращать одно значение (скалярный подзапрос), набор значений или использоваться для проверки существования строк. Общие табличные выражения (CTE), задаваемые в предложении `WITH`, позволяют дать подзапросу имя, вычислить его один раз и использовать результат в основном запросе.

Запрос 13. Скалярный подзапрос в условии `WHERE`: сначала вычисляется средняя мощность автомобилей автопарка, затем выбираются автомобили с большей мощностью.

```
SELECT *
FROM car
WHERE horsepower > (SELECT AVG(horsepower) FROM car);
```

Таблица 13 – Результат запроса 13

id	title	photo_url	description	horsepower	production_year	color	suspension_name	is_utilized
4	Toyota Supra	https://cars.pics/toyota_supra.jpg	Мотор 2JZ, тюнинг без границ. Такую вы видели только на картинках!	330	1997	Оранжевый	Double Wishbone	false
5	Nissan Skyline GT-R	https://cars.pics/nissan_skyline_gtr.jpg	Его знает каждый. Столько игр и фильмов ему посвящено!	280	1999	Синий	Multi-Link	false
7	BMW M3	https://cars.pics/bmw_m3.jpg	Рядная шестёрка, задний привод. Седан, а едет как спорткар.	510	2021	Серый	Adaptive M Suspension	false
10	Porsche 911	https://cars.pics/porsche_911.jpg	Оппозитник сзади. Уже 60 лет не меняют форму кузова. Могут, а зачем?.	450	2022	Чёрный	Sport Suspension	false
11	Audi RS2	https://cars.pics/audi_rs2.jpg	Универсал, который доводили сами инженеры из Porsche. Первый RS от Audi.	315	1994	Синий	Sport Suspension	false

Запрос 14. Коррелированный подзапрос с предикатом `NOT EXISTS`: выбираются тренеры, для которых в таблице `achievement` нет ни одной записи.

```
SELECT
    t.id,
    t.last_name,
    t.first_name
FROM trainer t
WHERE NOT EXISTS (
    SELECT 1
    FROM achievement a
    WHERE a.trainer_id = t.id
);
```

Таблица 14 – Результат запроса 14

id	last_name	first_name
8	Соколова	Анна

Не имеет достижений в соревнованиях один тренер — Соколова Анна.

Запрос 15. Общее табличное выражение `trainer_results` подсчитывает количество достижений каждого тренера. Далее результат соединяется с таблицей `trainer` для получения фамилий и имён.

```
WITH trainer_results AS (
    SELECT
        trainer_id,
        COUNT(*) AS achievement_count
    FROM achievement
    GROUP BY trainer_id
)
SELECT
    t.last_name,
    t.first_name,
    tr.achievement_count
FROM trainer_results tr
INNER JOIN trainer t ON t.id = tr.trainer_id
ORDER BY tr.achievement_count DESC;
```

Таблица 15 – Результат запроса 15

last_name	first_name	achievement_count
Моралес	Даниэль	2
Пехота	Наталья	2
Торрето	Доминик	2
Иванова	Мария	1
Иванов	Иван	1
Смирнов	Алексей	1
Петров	Дмитрий	1

3.4 Предложение RETURNING

Команды `INSERT`, `UPDATE` и `DELETE` могут возвращать затронутые строки с помощью предложения `RETURNING`. Для `UPDATE` возвращаются уже изменённые значения.

Запрос 16. Добавление нового автомобиля. Предложение `RETURNING *` возвращает созданную запись целиком, в том числе автоматически сгенерированный идентификатор и значения по умолчанию.

```sql
INSERT INTO car (
    title,
    description,
    horsepower,
    production_year,
    color,
    suspension_name
)
VALUES (
    'Volkswagen Beetle',
    'Самый стильный автомобиль в клубе',
    111,
    1975,
    'Красный',
    'MacPherson'
)
RETURNING *;
```

Таблица 16 – Результат запроса 16

id	title	photo_url	description	horsepower	production_year	color	suspension_name	is_utilized
15	Volkswagen Beetle	null	Самый стильный автомобиль в клубе	111	1975	Красный	MacPherson	false

Запрос 17. Увеличение мощности автомобиля на 20 л.с. Возвращаются только изменённые значения выбранных столбцов.

```sql
UPDATE car
SET horsepower = horsepower + 20
WHERE id = 3
RETURNING id, title, horsepower;
```

Таблица 17 – Результат запроса 17

id	title	horsepower
3	Mazda MX-5 Miata	221

Запрос 18. Удаление специализации тренера Торрето на автомобиле Nissan Skyline GT-R. Идентификаторы тренера и автомобиля определяются подзапросами, удалённая строка возвращается в результате.

```
DELETE FROM trainer_car
WHERE trainer_id IN (SELECT id FROM trainer WHERE last_name = 'Торрето')
    AND car_id IN (SELECT id FROM car WHERE title = 'Nissan Skyline GT-R')
RETURNING *;
```

Таблица 18 – Результат запроса 18

trainer_id	car_id
1	5

3.5 Планы выполнения запросов (EXPLAIN)

План выполнения запроса представляет собой дерево узлов, для каждого из которых выводятся оценка стоимости запуска и общая стоимость (`cost`), ожидаемое число строк (`rows`) и ширина строки (`width`). При использовании параметра `ANALYZE` запрос реально выполняется, и к оценкам добавляются фактическое время и число строк.

План 1. Обычный `EXPLAIN` для поиска автомобиля по названию.

```
EXPLAIN
SELECT *
FROM car
WHERE title = 'Mazda MX-5 Miata';
```

```
Index Scan using car_title_idx on car  (cost=0.14..8.16 rows=1 width=1059)
Index Cond: ((title)::text = 'Mazda MX-5 Miata'::text)
```

По столбцу `title` создан индекс, поэтому планировщик выбрал метод доступа *Index Scan* с условием поиска по индексу `car_title_idx`. Общая оценка стоимости составляет 8.16.

План 2. `EXPLAIN (ANALYZE)` для запроса с `NOT EXISTS`. Помимо оценок выводится фактическое время выполнения.

```sql
EXPLAIN (ANALYSE)
SELECT
    t.id,
    t.last_name,
    t.first_name
FROM trainer t
WHERE NOT EXISTS (
    SELECT 1
    FROM achievement a
    WHERE a.trainer_id = t.id
);
```

```
Nested Loop Anti Join  (cost=0.15..32.62 rows=30 width=444) (actual time=0.032..0.034 rows=1.00 loops=1)
Buffers: shared hit=16
->  Seq Scan on trainer t  (cost=0.00..10.60 rows=60 width=444) (actual time=0.014..0.015 rows=8.00 loops=1)
Buffers: shared hit=1
->  Index Only Scan using achievement_trainer_id_idx on achievement a  (cost=0.15..1.02 rows=4 width=8) (actual
↪  time=0.002..0.002 rows=0.88 loops=8)
Index Cond: (trainer_id = t.id)
Heap Fetches: 7
Index Searches: 8
Buffers: shared hit=15
Planning Time: 0.111 ms
Execution Time: 0.048 ms
```

Для соединения использован узел *Nested Loop Anti Join*: таблица `trainer` читается последовательно (*Seq Scan*), а для каждого тренера выполняется поиск в индексе `achievement_trainer_id_idx` (*Index Only Scan*). Время планирования составило 0.111 мс, время выполнения — 0.048 мс. Оценка числа строк таблицы `trainer` (60) заметно отличается от фактической (8), что объясняется малым объёмом данных и отсутствием собранной статистики.

План 3. `EXPLAIN (ANALYZE, BUFFERS)` с получением только столбца `title`. Поскольку этот столбец входит в индекс, применяется метод доступа *Index Only Scan*.

```sql
EXPLAIN (ANALYZE, BUFFERS)
SELECT title
FROM car
WHERE title = 'Toyota Supra';
```

```
Index Only Scan using car_title_idx on car  (cost=0.14..8.16 rows=1 width=218) (actual time=0.022..0.022
↪  rows=1.00 loops=1)
Index Cond: (title = 'Toyota Supra'::text)
```

```
Heap Fetches: 1
Index Searches: 1
Buffers: shared hit=2
Planning Time: 0.054 ms
Execution Time: 0.032 ms
```

Параметр `BUFFERS` добавил информацию об использовании буферов: `shared hit=2` означает, что оба нужных блока найдены в кеше, и чтения с диска не потребовалось. Значение `Heap Fetches: 1` показывает, что для найденной строки пришлось обратиться к таблице, так как страница ещё не отмечена в карте видимости (таблица недавно изменялась).

3.6 План запроса в формате JSON для нескольких связанных таблиц

План 4. Формат `JSON` содержит ту же информацию, что и текстовый, но удобнее для программной обработки. Выборка объединяет три связанные таблицы: `trainer`, `trainer_car` и `car`.

```
EXPLAIN (ANALYSE, FORMAT JSON)
SELECT
    t.last_name,
    t.first_name,
    c.title AS car_title
FROM trainer t
JOIN trainer_car tc ON tc.trainer_id = t.id
JOIN car c ON c.id = tc.car_id;
```

Результат выполнения:

```
[
  {
    "Plan": {
      "Node Type": "Hash Join",
      "Parallel Aware": false,
      "Async Capable": false,
      "Join Type": "Inner",
      "Startup Cost": 22.92,
      "Total Cost": 61.34,
      "Plan Rows": 1850,
      "Plan Width": 654,
      "Actual Startup Time": 0.045,
      "Actual Total Time": 0.051,
      "Actual Rows": 17.00,
      "Actual Loops": 1,
      "Disabled": false,
      "Inner Unique": true,
      "Hash Cond": "(tc.car_id = c.id)",
      "Shared Hit Blocks": 3,
```

```
"Shared Read Blocks": 0,
"Shared Dirtied Blocks": 0,
"Shared Written Blocks": 0,
"Local Hit Blocks": 0,
"Local Read Blocks": 0,
"Local Dirtied Blocks": 0,
"Local Written Blocks": 0,
"Temp Read Blocks": 0,
"Temp Written Blocks": 0,
"Plans": [
  {
    "Node Type": "Hash Join",
    "Parent Relationship": "Outer",
    "Parallel Aware": false,
    "Async Capable": false,
    "Join Type": "Inner",
    "Startup Cost": 11.35,
    "Total Cost": 44.81,
    "Plan Rows": 1850,
    "Plan Width": 444,
    "Actual Startup Time": 0.019,
    "Actual Total Time": 0.022,
    "Actual Rows": 17.00,
    "Actual Loops": 1,
    "Disabled": false,
    "Inner Unique": true,
    "Hash Cond": "(tc.trainer_id = t.id)",
    "Shared Hit Blocks": 2,
    "Shared Read Blocks": 0,
    "Shared Dirtied Blocks": 0,
    "Shared Written Blocks": 0,
    "Local Hit Blocks": 0,
    "Local Read Blocks": 0,
    "Local Dirtied Blocks": 0,
    "Local Written Blocks": 0,
    "Temp Read Blocks": 0,
    "Temp Written Blocks": 0,
    "Plans": [
      {
        "Node Type": "Seq Scan",
        "Parent Relationship": "Outer",
        "Parallel Aware": false,
        "Async Capable": false,
        "Relation Name": "trainer_car",
        "Alias": "tc",
        "Startup Cost": 0.00,
        "Total Cost": 28.50,
        "Plan Rows": 1850,
        "Plan Width": 16,
        "Actual Startup Time": 0.005,
        "Actual Total Time": 0.006,
        "Actual Rows": 17.00,
        "Actual Loops": 1,
        "Disabled": false,
        "Shared Hit Blocks": 1,
        "Shared Read Blocks": 0,
        "Shared Dirtied Blocks": 0,
        "Shared Written Blocks": 0,
        "Local Hit Blocks": 0,
        "Local Read Blocks": 0,
        "Local Dirtied Blocks": 0,
        "Local Written Blocks": 0,
        "Temp Read Blocks": 0,
        "Temp Written Blocks": 0
      },
      {
        "Node Type": "Hash",
        "Parent Relationship": "Inner",
        "Parallel Aware": false,
        "Async Capable": false,
        "Startup Cost": 10.60,
        "Total Cost": 10.60,
        "Plan Rows": 60,
        "Plan Width": 444,
        "Actual Startup Time": 0.008,
        "Actual Total Time": 0.008,
        "Actual Rows": 8.00,
        "Actual Loops": 1,
```

```
      "Disabled": false,
      "Hash Buckets": 1024,
      "Original Hash Buckets": 1024,
      "Hash Batches": 1,
      "Original Hash Batches": 1,
      "Peak Memory Usage": 9,
      "Shared Hit Blocks": 1,
      "Shared Read Blocks": 0,
      "Shared Dirtied Blocks": 0,
      "Shared Written Blocks": 0,
      "Local Hit Blocks": 0,
      "Local Read Blocks": 0,
      "Local Dirtied Blocks": 0,
      "Local Written Blocks": 0,
      "Temp Read Blocks": 0,
      "Temp Written Blocks": 0,
      "Plans": [
        {
          "Node Type": "Seq Scan",
          "Parent Relationship": "Outer",
          "Parallel Aware": false,
          "Async Capable": false,
          "Relation Name": "trainer",
          "Alias": "t",
          "Startup Cost": 0.00,
          "Total Cost": 10.60,
          "Plan Rows": 60,
          "Plan Width": 444,
          "Actual Startup Time": 0.005,
          "Actual Total Time": 0.006,
          "Actual Rows": 8.00,
          "Actual Loops": 1,
          "Disabled": false,
          "Shared Hit Blocks": 1,
          "Shared Read Blocks": 0,
          "Shared Dirtied Blocks": 0,
          "Shared Written Blocks": 0,
          "Local Hit Blocks": 0,
          "Local Read Blocks": 0,
          "Local Dirtied Blocks": 0,
          "Local Written Blocks": 0,
          "Temp Read Blocks": 0,
          "Temp Written Blocks": 0
        }
      ]
    }
  ]
},
{
  "Node Type": "Hash",
  "Parent Relationship": "Inner",
  "Parallel Aware": false,
  "Async Capable": false,
  "Startup Cost": 10.70,
  "Total Cost": 10.70,
  "Plan Rows": 70,
  "Plan Width": 226,
  "Actual Startup Time": 0.019,
  "Actual Total Time": 0.020,
  "Actual Rows": 12.00,
  "Actual Loops": 1,
  "Disabled": false,
  "Hash Buckets": 1024,
  "Original Hash Buckets": 1024,
  "Hash Batches": 1,
  "Original Hash Batches": 1,
  "Peak Memory Usage": 9,
  "Shared Hit Blocks": 1,
  "Shared Read Blocks": 0,
  "Shared Dirtied Blocks": 0,
  "Shared Written Blocks": 0,
  "Local Hit Blocks": 0,
  "Local Read Blocks": 0,
  "Local Dirtied Blocks": 0,
  "Local Written Blocks": 0,
  "Temp Read Blocks": 0,
  "Temp Written Blocks": 0,
  "Plans": [
```

```
          {
            "Node Type": "Seq Scan",
            "Parent Relationship": "Outer",
            "Parallel Aware": false,
            "Async Capable": false,
            "Relation Name": "car",
            "Alias": "c",
            "Startup Cost": 0.00,
            "Total Cost": 10.70,
            "Plan Rows": 70,
            "Plan Width": 226,
            "Actual Startup Time": 0.013,
            "Actual Total Time": 0.015,
            "Actual Rows": 12.00,
            "Actual Loops": 1,
            "Disabled": false,
            "Shared Hit Blocks": 1,
            "Shared Read Blocks": 0,
            "Shared Dirtied Blocks": 0,
            "Shared Written Blocks": 0,
            "Local Hit Blocks": 0,
            "Local Read Blocks": 0,
            "Local Dirtied Blocks": 0,
            "Local Written Blocks": 0,
            "Temp Read Blocks": 0,
            "Temp Written Blocks": 0
          }
        ]
      }
    ]
  },
  "Planning": {
    "Shared Hit Blocks": 0,
    "Shared Read Blocks": 0,
    "Shared Dirtied Blocks": 0,
    "Shared Written Blocks": 0,
    "Local Hit Blocks": 0,
    "Local Read Blocks": 0,
    "Local Dirtied Blocks": 0,
    "Local Written Blocks": 0,
    "Temp Read Blocks": 0,
    "Temp Written Blocks": 0
  },
  "Planning Time": 0.162,
  "Triggers": [
  ],
  "Execution Time": 0.069
 }
]
```


Корневой узел плана — *Hash Join*, общая стоимость 61.34, фактическое время 0.051 мс. Внутри него расположен второй *Hash Join*, соединяющий `trainer_car` и `trainer`; все три таблицы читаются последовательно (*Seq Scan*). В результате получено 17 строк, время планирования составило 0.162 мс, время выполнения — 0.069 мс.

3.7 Способы соединения наборов строк

Для одной и той же выборки (тренеры и автомобили, на которых они специализируются) выполнено соединение тремя способами: вложенным циклом (*Nested Loop*), хешированием (*Hash Join*) и слиянием (*Merge Join*). Нужный способ включается принудительно: параметрами `enable_nestloop`,

enable_hashjoin и enable_mergejoin отключаются остальные способы. Для каждого варианта получен план в формате JSON с параметром ANALYZE. Ниже приведено сокращённое представление плана (тип узла, стоимость, фактическое время и число строк) и основные показатели.

Вариант 1. Вложенный цикл.

```
SET enable_nestloop = ON;
SET enable_hashjoin = OFF;
SET enable_mergejoin = OFF;

EXPLAIN (ANALYZE, FORMAT JSON)
SELECT
    t.id,
    t.last_name,
    c.title
FROM trainer t
INNER JOIN trainer_car tc ON tc.trainer_id = t.id
INNER JOIN car c ON c.id = tc.car_id;
```

```
Nested Loop  (cost=0.29..425.70, actual time=0.037..0.077, rows=17)
   -> Nested Loop  (cost=0.15..86.70, actual time=0.030..0.057, rows=17)
      -> Seq Scan on car  (cost=0.00..10.70, actual time=0.014..0.015, rows=12)
      -> Index Scan on trainer_car using trainer_car_car_id_idx  (cost=0.15..1.00, actual
      ↪  time=0.002..0.003, rows=1)
   -> Index Scan on trainer using trainer_pkey  (cost=0.14..0.18, actual time=0.001..0.001,
   ↪  rows=1)
```

Из таблицы car (12 строк) выполняется последовательное чтение, а для каждой строки поиск соответствующих записей в trainer_car и trainer выполняется по индексам trainer_car_car_id_idx и trainer_pkey. Большое число обращений к индексам даёт самую высокую стоимость плана.

Вариант 2. Хеширование.

```
SET enable_nestloop = OFF;
SET enable_hashjoin = ON;
SET enable_mergejoin = OFF;

EXPLAIN (ANALYZE, FORMAT JSON)
SELECT
```

```
    t.id,
    t.last_name,
    c.title
FROM trainer t
JOIN trainer_car tc ON tc.trainer_id = t.id
JOIN car c ON c.id = tc.car_id;
```

```
Hash Join  (cost=22.92..61.34, actual time=0.053..0.061, rows=17)
   -> Hash Join  (cost=11.35..44.81, actual time=0.022..0.026, rows=17)
      -> Seq Scan on trainer_car  (cost=0.00..28.50, actual time=0.006..0.007, rows=17)
      -> Hash  (cost=10.60..10.60, actual time=0.010..0.010, rows=8)
         -> Seq Scan on trainer  (cost=0.00..10.60, actual time=0.006..0.007, rows=8)
   -> Hash  (cost=10.70..10.70, actual time=0.023..0.024, rows=12)
      -> Seq Scan on car  (cost=0.00..10.70, actual time=0.017..0.018, rows=12)
```

Меньшие наборы строк помещаются в хеш-таблицы, после чего большая таблица `trainer_car` читается один раз последовательно. Подготовка хеш-таблиц дешёвая, так как таблицы небольшие.

Вариант 3. Слияние.

```
SET enable_nestloop = OFF;
SET enable_hashjoin = OFF;
SET enable_mergejoin = ON;

EXPLAIN (ANALYZE, FORMAT JSON)
SELECT
    t.id,
    t.last_name,
    c.title
FROM trainer t
JOIN trainer_car tc ON tc.trainer_id = t.id
JOIN car c ON c.id = tc.car_id;
```

```
Merge Join  (cost=211.99..228.07, actual time=0.051..0.055, rows=17)
   -> Sort  (cost=199.14..203.77, actual time=0.041..0.042, rows=17)
      -> Merge Join  (cost=12.52..98.75, actual time=0.030..0.037, rows=17)
         -> Index Only Scan on trainer_car using trainer_car_pkey  (cost=0.15..75.90, actual
         ↪  time=0.011..0.015, rows=17)
         -> Sort  (cost=12.37..12.52, actual time=0.016..0.016, rows=8)
            -> Seq Scan on trainer  (cost=0.00..10.60, actual time=0.010..0.011, rows=8)
   -> Sort  (cost=12.85..13.02, actual time=0.009..0.009, rows=11)
      -> Seq Scan on car  (cost=0.00..10.70, actual time=0.005..0.005, rows=12)
```

Для слияния оба набора должны быть отсортированы по ключу соединения, поэтому в плане появились узлы *Sort*, а общая стоимость выросла за счёт сортировки. Для таблицы `trainer_car` использован *Index Only Scan* по первичному ключу.

Показатели трёх вариантов сведены в таблицу 19: *Actual Time* — фактическое время корневого узла, *Planning* и *Execution* — время планирования и выполнения запроса, *Shared Hit* — число блоков, найденных в разделяемом буфере.

Таблица 19 – Сравнение способов соединения

Способ	**Total Cost**	**Actual Time, мс**	**Planning, мс**	**Execution, мс**	**Shared Hit**
Nested Loop	425.70	0.077	0.181	0.101	59
Hash Join	61.34	0.061	0.181	0.080	3
Merge Join	228.07	0.055	0.159	0.079	4

Результаты показывают, что наименьшую оценку стоимости (61.34) имеет соединение хешированием, самую высокую (425.70) — вложенный цикл. Время выполнения всех трёх вариантов составляет порядка 0.08–0.10 мс; на выборке из 17 строк разница между ними сопоставима с погрешностью измерения, при этом вложенный цикл оказался самым медленным и обратился к наибольшему числу буферов (59 против 3 у хеширования и 4 у слияния). Планировщик при включённых по умолчанию способах соединения сам выбрал *Hash Join* (см. план 4). Таким образом, для данной выборки наиболее оптимальным способом является **соединение хешированием (Hash Join)**: оно имеет наименьшую стоимость и наименьшее число обращений к буферам, а время выполнения практически не отличается от лучшего (0.080 мс у хеширования и 0.079 мс у слияния). При росте объёмов данных это преимущество будет увеличиваться, так как хеширование читает каждую таблицу один раз, а вложенный цикл выполняет обращение к индексу для каждой строки внешнего набора.

Вывод

В ходе выполнения лабораторной работы были освоены агрегатные функции с группировкой строк и фильтрацией групп (`GROUP BY`, `HAVING`), операторы и встроенные функции PostgreSQL, работа с датами и временными интервалами, подзапросы, общие табличные выражения и предложение `RETURNING`. Все примеры из методических указаний выполнены для базы данных гоночного автоклуба.

Получены и проанализированы планы выполнения запросов с параметрами `ANALYZE` и `BUFFERS`, а также в формате `JSON` для выборки из нескольких связанных таблиц. Рассмотрены методы доступа *Index Scan* и *Index Only Scan*. Три способа соединения (*Nested Loop*, *Hash Join*, *Merge Join*) сравнены на одной выборке; наиболее оптимальным признано соединение хешированием.